

✓ EDA Notebook

✓ 0. Setup Environment

```
# DO NOT MODIFY THE CODE IN THIS CELL
!pip install -q utstd

from utstd.folders import *
from utstd.ipyrenders import *

at = AtFolder(
    course_code=36106,
    assignment="AT3",
)
at.run()

import warnings
warnings.simplefilter(action='ignore')
```

```
12.9/12.9 MB 47.8 MB/s eta 0s
4.9/4.9 MB 56.6 MB/s eta 0s
ERROR: pip's dependency resolver does not currently take into account
umap-learn 0.5.12 requires scikit-learn>=1.6, but you have scikit-learn
hdbscan 0.8.42 requires scikit-learn>=1.6, but you have scikit-learn 1
Mounted at /content/gdrive
```

You can now save your data files in: /content/gdrive/MyDrive/36106/ass

✓ Student Information

```
group_name = "36106-26AU-AT3-Group 20"
student_name = "Devvrat Charusmiti Joshi"
student_id = "25657887"
```

```
# Do not modify this code
print_tile(size="h1", key='group_name', value=group_name)
```

group_name

36106-26AU-AT3-Group 20

```
# Do not modify this code
print_tile(size="h1", key='student_name', value=student_name)
```

student_name

Devvrat Charusmiti Joshi

```
# Do not modify this code
print_tile(size="h1", key='student_id', value=student_id)
```

student_id

25657887

✓ 0. Python Packages

0.a Install Additional Packages

If you are using additional packages, you need to install them here using the command: `! pip install <package_name>`

✓ 0.b Import Packages

```
import pandas as pd
import altair as alt
import numpy as np

# Display settings
pd.set_option("display.max_columns", None)
pd.set_option("display.max_rows", 100)

# Allow Altair to display larger datasets
alt.data_transformers.disable_max_rows()

DataTransformerRegistry.enable('default')
```

✓ B. Data Understanding

```
# Do not modify this code
try:
```

```
df = pd.read_csv(at.folder_path / "person.csv")
except Exception as e:
    print(e)
```

✓ B.1 Explore Dataset

```
df.head()
```

	person_id	name_style	email_promotion	person_type	title	first_n
0	0e7956c1-835b-487b-b4b3-b83d08811282	0.0	0.0	IN	NaN	C
1	333a1b3e-04c5-4c0f-80db-d840f5cc6593	0.0	1.0	SC	Mr.	

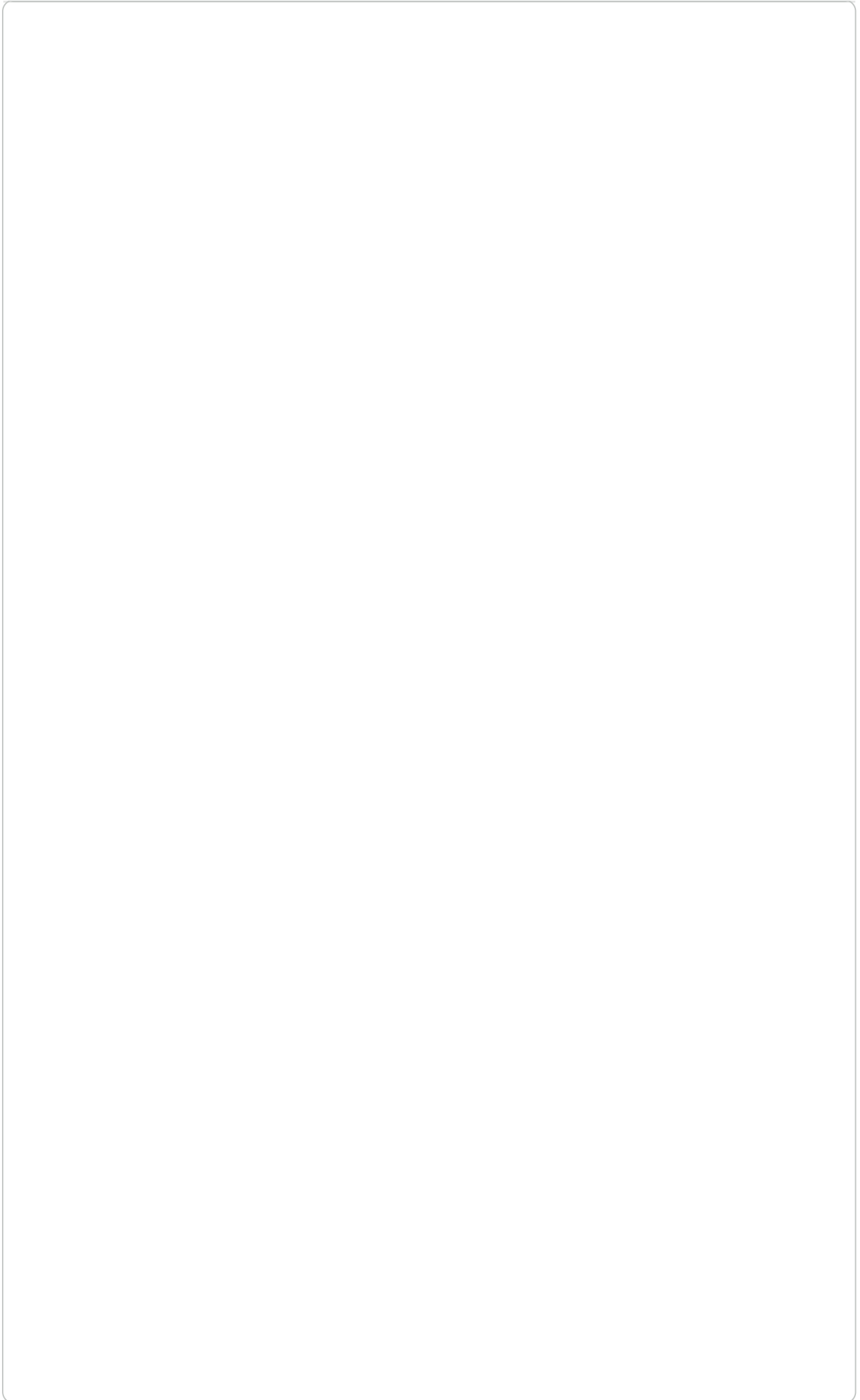
```
# Basic dataset information
print("Dataset shape:", df.shape)
print("\nColumn names:")
print(df.columns.tolist())

print("\nData types:")
display(df.dtypes.to_frame(name="data_type"))

print("\nFirst 5 rows:")
display(df.head())

print("\nSummary statistics for numerical columns:")
display(df.describe(include=[np.number]))

print("\nSummary statistics for categorical columns:")
display(df.describe(include=["object"]))
```



Dataset shape: (25163, 10)

Column names:
['person_id', 'name_style', 'email_promotion', 'person_type', 'title']

Data types:

	data_type
person_id	object

```
# Missing value analysis
missing_summary = pd.DataFrame({
    "missing_count": df.isna().sum(),
    "missing_percentage": (df.isna().mean() * 100).round(2),
    "non_missing_count": df.notna().sum()
}).sort_values(by="missing_percentage", ascending=False)

missing_summary
```

	middle_name	missing_count	missing_percentage	non_missing_count
	additional_contact_info	25152	99.96	
	suffix	25098	99.74	
	title	23888	94.93	1275
First 5 rows	middle_name	10757	42.75	14406
	email_promotion	210	0.83	24953
	name_style	73	0.29	25090
	person_type	35	0.14	25128
	person_id	0	0.00	25163
	first_name	0	0.00	25163
	last_name	0	0.00	25163

```
# Duplicate analysis
total_rows = len(df)
duplicate_rows = df.duplicated().sum()
unique_person_ids = df["person_id"].nunique()
duplicate_person_ids = total_rows - unique_person_ids

duplicate_summary = pd.DataFrame({
    "metric": [
        "Total rows",
        "Fully duplicated rows",
        "Unique person_id values",
        "Rows beyond unique person_id count"
    ],
    "value": [
        total_rows,
        duplicate_rows,
        unique_person_ids,
        duplicate_person_ids
    ]
})
```

```

        duplicate_person_ids
    ]
})

```

duplicate_summary

	50%	0.0	metric	value
0			Total rows	25163
1			Fully duplicated rows	5191
2			Unique person_id values	19972
3			Rows beyond unique person_id count	5191

```

# Check repeated person_id values
person_id_duplicates = (
    df["person_id"]
    .value_counts()
    .reset_index()
)

person_id_duplicates.columns = ["person_id", "count"]

person_id_duplicates.head(10)

```

	person_id	count
0	d2ac535d-8a05-44d5-bc15-c72e1747866c	4
1	12d41b79-d86b-42ce-9a12-508ccc55f1a9	4
2	ddd645a2-7a2f-4fcd-9cc2-df4be60c30c0	4
3	fde8d39e-6b8c-4441-a78d-144658727b88	4
4	0a7a4592-bc7c-47f5-b6c1-e244de008ec8	4
5	2c09e9bb-f93d-49ca-9994-a1623475f6df	4
6	84d1c0c9-24a9-4f51-b104-7d6625b4b5d6	4
7	feef6c6-4a93-49dc-9902-c6de5c7199e4	4
8	c9dddb21-6b41-4819-b3ae-ffd83218f871	4
9	f1673f6d-fbbc-4e80-8b06-efa20d0b03b9	4

```

# Visualise missing percentage by column
missing_chart_data = missing_summary.reset_index().rename(columns={"i

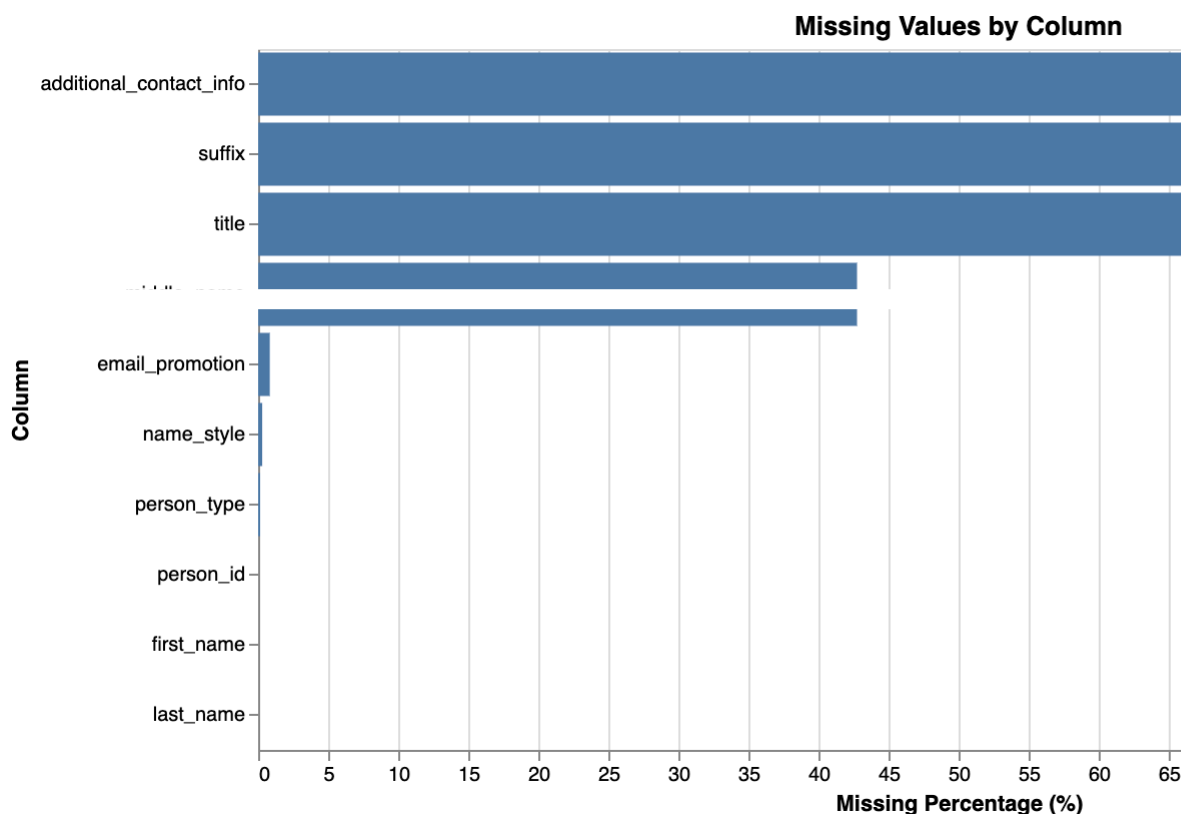
alt.Chart(missing_chart_data).mark_bar().encode(
    x=alt.X("missing_percentage:Q", title="Missing Percentage (%)",
    y=alt.Y("column:N", sort="-x", title="Column"),
    tooltip=["column", "missing_count", "missing_percentage"]
).properties(

```

```

title="Missing Values by Column",
width=700,
height=350
)

```



```
dataset_insights = ""
```

The person dataset contains 25,163 rows and 10 columns. The main identifier is person_id, which can be used later to join this dataset with customer, sales, or order-related datasets. The dataset contains personal information such as name fields, person type, email promotion preference, and additional contact information. The dataset has an important duplicate issue. There are 5,191 fully duplicated rows, and only 19,972 unique person_id values. This means duplicate records should be removed before using this dataset for modelling or joining with other datasets. If duplicates

```
# Do not modify this code
```

```
print_tile(size="h3", key='dataset_insights', value=dataset_insights)
```

```
dataset_insights
```

The person dataset contains 25,163 rows and 10 columns. The main identifier is person_id, which can be used later to join this dataset with customer, sales, or order-related datasets. The dataset contains personal information such as name fields, person type, email promotion preference, and additional contact information. The dataset has an important duplicate issue. There are 5,191 fully duplicated rows, and only 19,972 unique person_id values. This means duplicate records should be removed before using this dataset for modelling or joining with other datasets. If duplicates

✓ B.2 Explore Feature of Interest person_type

```

# Explore person_type distribution
person_type_summary = (

```

```

df["person_type"]
    .value_counts(dropna=False)
    .reset_index()
)

person_type_summary.columns = ["person_type", "count"]
person_type_summary["percentage"] = (
    person_type_summary["count"] / len(df) * 100
).round(2)

person_type_summary

```

	person_type	count	percentage
0	IN	23229	92.31
1	SC	955	3.80
2	GC	371	1.47
3	EM	359	1.43
4	VC	192	0.76
5	NaN	35	0.14
6	SP	22	0.09

```

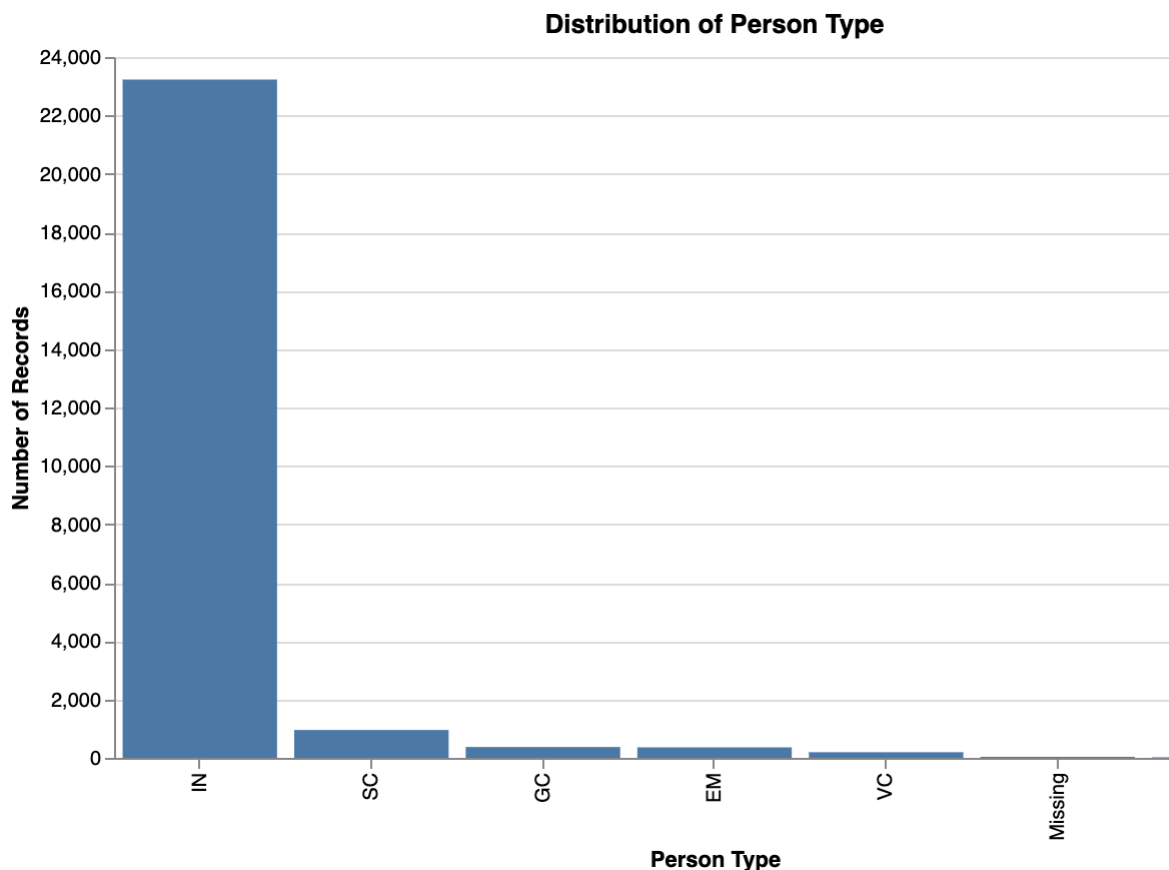
# Replace missing values for visualisation only
person_type_chart_data = df.copy()
person_type_chart_data["person_type"] = person_type_chart_data["person_type"].fillna("NaN")

person_type_counts = (
    person_type_chart_data["person_type"]
    .value_counts()
    .reset_index()
)

person_type_counts.columns = ["person_type", "count"]

alt.Chart(person_type_counts).mark_bar().encode(
    x=alt.X("person_type:N", title="Person Type", sort="-y"),
    y=alt.Y("count:Q", title="Number of Records"),
    tooltip=["person_type", "count"]
).properties(
    title="Distribution of Person Type",
    width=600,
    height=350
)

```

```
feature_1_insights = ""
```

The person_type feature categorises each person into a type. The most common value is IN, which accounts for the vast majority of records. Other person types such as SC, GC, EM, VC, and SP appear much less frequently. This shows that the dataset is highly imbalanced across person types. For the regression use case, person_type may be useful because different person categories may have different purchasing behaviour or business roles. For example, some person types may represent individual customers, store contacts, employees, vendor contacts, or sales-related

```
# Do not modify this code
```

```
print_tile(size="h3", key='feature_1_insights', value=feature_1_insights)
```

```
feature_1_insights
```

The person_type feature categorises each person into a type. The most common value is IN, which accounts for the vast majority of records. Other person types such as SC, GC, EM, VC, and SP appear much less frequently. This shows that the dataset is highly imbalanced across person types. For the regression use case, person_type may be useful because different person categories may have different purchasing behaviour or business roles. For example, some person types may represent individual customers, store contacts, employees, vendor contacts, or sales-related

✓ B.3 Explore Feature of Interest email_promotion

```
# Explore email_promotion distribution
email_promotion_summary = (
    df["email_promotion"]
    .value_counts(dropna=False)
    .reset_index()
```

```
)

email_promotion_summary.columns = ["email_promotion", "count"]
email_promotion_summary["percentage"] = (
    email_promotion_summary["count"] / len(df) * 100
).round(2)

email_promotion_summary
```

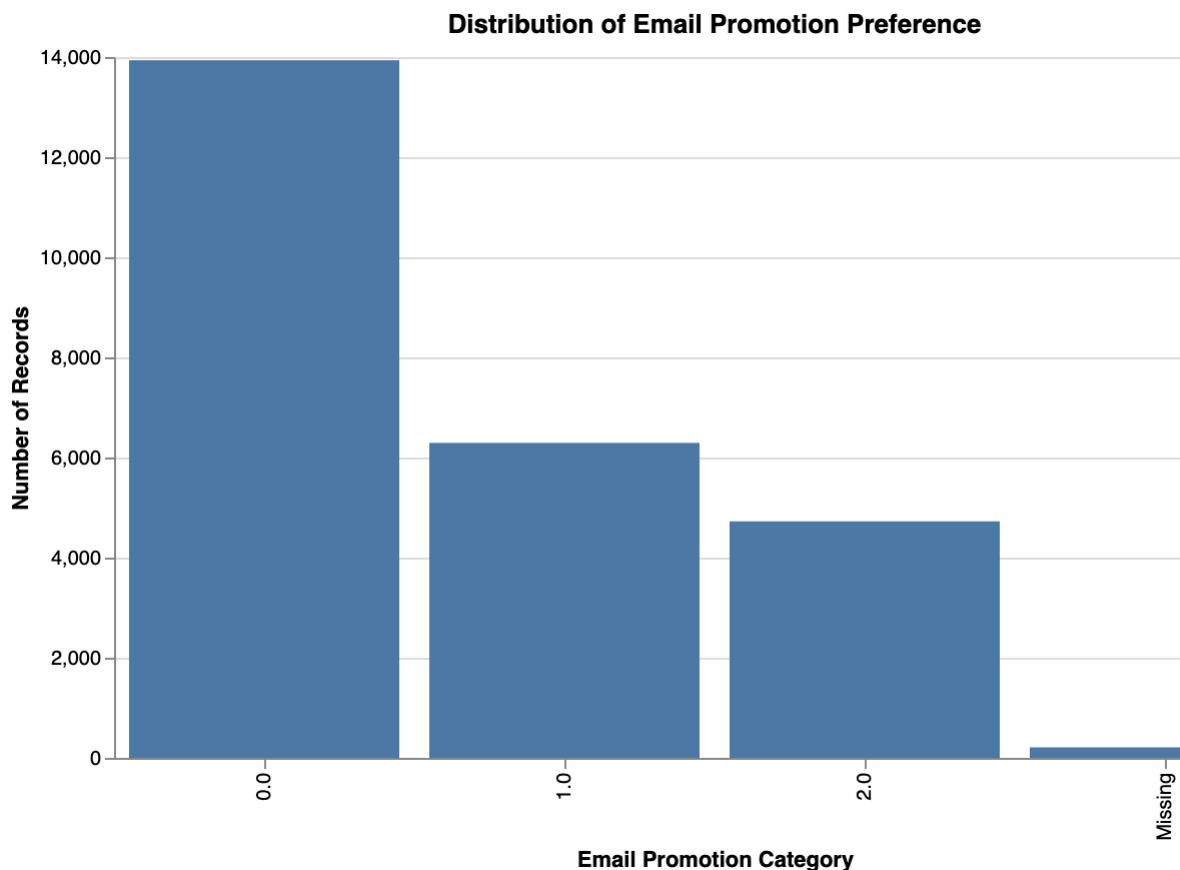
	email_promotion	count	percentage
0	0.0	13936	55.38
1	1.0	6292	25.00
2	2.0	4725	18.78
3	NaN	210	0.83

```
# Prepare email_promotion for visualisation
email_chart_data = df.copy()
email_chart_data["email_promotion"] = email_chart_data["email_promotion"]
email_chart_data["email_promotion"] = email_chart_data["email_promotion"]

email_promotion_counts = (
    email_chart_data["email_promotion"]
    .value_counts()
    .reset_index()
)

email_promotion_counts.columns = ["email_promotion", "count"]

alt.Chart(email_promotion_counts).mark_bar().encode(
    x=alt.X("email_promotion:N", title="Email Promotion Category", so
    y=alt.Y("count:Q", title="Number of Records"),
    tooltip=["email_promotion", "count"]
).properties(
    title="Distribution of Email Promotion Preference",
    width=600,
    height=350
)
```



```
feature_2_insights = ""
```

The email_promotion feature shows the promotional email preference of This feature may be useful for the regression use case because custom The feature has a small number of missing values. These can be filled

```
# Do not modify this code
```

```
print_tile(size="h3", key='feature_2_insights', value=feature_2_insights)
```

```
feature_2_insights
```

The email_promotion feature shows the promotional email preference of each person. The values are mainly 0, 1, and 2, with 0 being the most common category. This suggests that many people either do not receive promotional emails or belong to the lowest promotional email category. This feature may be useful for the regression use case because customers who receive or accept promotional emails may have different purchasing behaviour compared with customers who do not. For example, customers with higher email promotion values may be more responsive to campaigns.

✓ B.4 Explore Feature of Interest name_style

```
# Explore name_style distribution
name_style_summary = (
    df["name_style"]
    .value_counts(dropna=False)
    .reset_index()
```

```
)

name_style_summary.columns = ["name_style", "count"]
name_style_summary["percentage"] = (
    name_style_summary["count"] / len(df) * 100
).round(2)

name_style_summary
```

	name_style	count	percentage
0	0.0	25090	99.71
1	NaN	73	0.29

```
# Prepare name_style for visualisation
name_style_chart_data = df.copy()
name_style_chart_data["name_style"] = name_style_chart_data["name_sty
name_style_chart_data["name_style"] = name_style_chart_data["name_sty

name_style_counts = (
    name_style_chart_data["name_style"]
    .value_counts()
    .reset_index()
)

name_style_counts.columns = ["name_style", "count"]

alt.Chart(name_style_counts).mark_bar().encode(
    x=alt.X("name_style:N", title="Name Style", sort="-y"),
    y=alt.Y("count:Q", title="Number of Records"),
    tooltip=["name_style", "count"]
).properties(
    title="Distribution of Name Style",
    width=600,
    height=350
)
```